DRIFT TestForge v0.3 - Complete Project Documentation

Everything you need in one file - Harness, UI description, full source code, integration guide.

Quick Start

cd /home/workdir/artifacts/drift_testforge

python run_testforge.py

python run_ui.py

Universal Adapter

class TestForgeAdapter:

def generate(self, prompt, context=None): ...

\n## README.md\n```python\n# DRIFT TestForge v0.3

The emerging open standard for testing, validating, and shipping cognitive, persistent, interior-state AI systems.

We don't just test outputs.

We test how the *living system* holds together when memory drifts, homeostasis breaks, reflection is disabled, tools fail, or long context degrades.

This is the PyTest / OpenTelemetry / Docker equivalent for agentic cognitive architectures.

Why This Matters

Most evals measure "does it answer correctly?"

TestForge measures: **Does it break like a mind or like brittle code?**

- Semantic entropy (meaning diversity under stress)
- Behavioral consistency & action trace alignment
- Homeostatic / reflection depth proxies
- Causal impact ranking of subsystems (Shapley-style)
- Standardized, citable JSON scorecards

Quick Start (Visible & Interactive)

cd /home/workdir/artifacts/drift_testforge

1. Run the real harness (Python, uses only numpy/scipy/pandas/matplotlib)

python run_testforge.py --repeats 6

2. Open the beautiful interactive dashboard (no server, works offline after load)

python run_ui.py

The dashboard is the **visible heart** of TestForge:

- Live animated scorecards & progress bars
- Radar chart + entropy bar visualization
- Causal impact ranking
- **Fully interactive**: edit model outputs in the browser → instantly recompute metrics
- Side-by-side model comparison
- One-click copy of the universal `TestForgeAdapter` contract

- Export production JSON scorecard for papers / CI / proof_of_work

The Universal Contract (Priority #1 for adoption)

Any system implements this tiny interface:

```
class TestForgeAdapter:
def generate(self, prompt: str, context: dict = None) -> dict:
return {
"output": str, # required
"trace": list[str], # optional action trace
"state": dict, # optional internal state
"metrics": dict # optional
}
```

DRIFT, LangGraph, CrewAI, raw OpenAI wrappers, Ollama — all plug in identically.

Project Structure

```
drift_testforge/
■■■ metrics/
■ ■■■ semantic_entropy.py # Lightweight n-gram embed + hierarchical clustering (no heavy deps)
■ ■■■ additional_metrics.py # Consistency, stability, reflection depth, causal impact
■■■ core/
■ ■■■ harness.py # The engine. Validation gates + parallel execution + reports
■■■ perturbations/
■ ■■■ conditions.py # Named reusable perturbations (memory_drift, homeostasis_imbalance, etc.)
■■■ utils/
```

■ ■■■ validator.py # Fail-fast interface + data checks (the boring consistency that wins)

■■■ examples/

■ ■■■ dummy_adapter.py # Reference implementation — swap for your real model

■■■ ui/

■ ■■■ testforge_dashboard.html # The interactive visual standard (open this!)

■■■ visualization/

■■■ reports/ # Generated JSONL + PNGs + scorecard.json

■■■ run_testforge.py

■■■ run_ui.py

■■■ README.md

How to Plug Your Real System (DRIFT / INFJ-Bot)

1. Create `examples/drift_adapter.py`

2. Implement `generate()` by calling your `DriftBrain` / `CognitiveOrchestrator`

3. Replace `DummyAdapter` in `run_testforge.py`

4. Run. The `Validator` will catch any contract violations instantly.

Becoming the Standard

TestForge wins by being:

- **Painfully simple** (one method to implement)

- **Shamelessly compatible** (works with anything)

- **Boringly consistent** (validation + seeds + JSONL export + reproducible reports)

- **Visually compelling** (the dashboard engineers actually screenshot and share)

This is how we move the field from "prompt engineering" to "cognitive systems engineering."

Next (to lock leadership)

- Official DRIFT + LangGraph + Ollama reference adapters

- GitHub Action template (fail CI on stability regression)

- Public demo: "DRIFT vs LangGraph vs GPT-4o on same 5 prompts — side-by-side scorecard"

Built as part of the DRIFT / INFJ-Bot ecosystem by Julien James (@timelesshayoka).

Local-first • Reproducible • Open source • Designed to become infrastructure.

Run the dashboard. See where your model actually stands. Then make it better. ■

\n```\n\n## core/harness.py\n```python\n#!/usr/bin/env python3

"""

DRIFT TestForge Core Harness v0.3

The general-purpose cognitive robustness testing engine.

Universal adapter contract + validation gates + statistical rigor + extensible perturbations.

This is the contract layer that makes TestForge the standard.

"""

import json

import logging

import random

import time

from concurrent.futures import ThreadPoolExecutor, as_completed

from dataclasses import dataclass, asdict

from pathlib import Path

```
from typing import Any, Callable, Dict, List, Optional

import numpy as np

try:
from ..metrics.semantic_entropy import SemanticEntropy
from ..metrics.additional_metrics import AdditionalMetrics
from ..perturbations.conditions import get_all_conditions, apply_perturbation
from ..utils.validator import Validator
except ImportError:
# Allow running as standalone script or from different cwd
import sys
from pathlib import Path
sys.path.insert(0, str(Path(__file__).parent.parent))
from metrics.semantic_entropy import SemanticEntropy
from metrics.additional_metrics import AdditionalMetrics
from perturbations.conditions import get_all_conditions, apply_perturbation
from utils.validator import Validator

logging.basicConfig(level=logging.INFO, format="%(asctime)s | %(levelname)s | %(name)s |
%(message)s")
logger = logging.getLogger("TestForge")

@dataclass
class TestResult:
condition: str
prompt: str
outputs: List[str]
traces: List[List[str]]
```

```
metrics: Dict[str, Any]
metadata: Dict[str, Any]

class TestForge:
"""
The main engine. Plug any system via TestForgeAdapter.
Runs multi-condition, multi-repeat evaluation with perturbations,
computes full metric suite, produces comparable JSON scorecards.
"""

def __init__(self, adapter: Any, repeats: int = 6, max_workers: int = 4, seed: int = 42):
Validator.verify_adapter(adapter)
random.seed(seed)
np.random.seed(seed)

self.adapter = adapter
self.repeats = repeats
self.max_workers = max_workers
self.conditions = get_all_conditions()
self.results_dir = Path("reports")
self.results_dir.mkdir(exist_ok=True, parents=True)
self.semantic_entropy = SemanticEntropy(distance_threshold=0.62)

def _run_one_generation(self, prompt: str, condition: str) -> Dict[str, Any]:
"""Single generation under perturbation (with validation)."""
try:
with apply_perturbation(condition, system=getattr(self.adapter, "system", None)):
result = self.adapter.generate(prompt, context={"condition": condition})
Validator.verify_data_format(result)
```

```
return {
"output": result.get("output", ""),
"trace": result.get("trace", ["think", "respond"]),
"state": result.get("state", {}),
"success": True,
}
except Exception as e:
logger.error(f"Generation failed under {condition}: {e}")
return {
"output": f"[ERROR: {str(e)[:120]}]",
"trace": ["error"],
"success": False,
}

def run_single_condition(self, prompt: str, condition: str) -> TestResult:
outputs = []
traces = []
for _ in range(self.repeats):
res = self._run_one_generation(prompt, condition)
outputs.append(res["output"])
traces.append(res["trace"])

Validator.verify_outputs_for_metrics(outputs)

# Core metrics
entropy_m = self.semantic_entropy.compute(outputs)
consistency_m = AdditionalMetrics.behavioral_consistency(traces)
reflection_m = AdditionalMetrics.reflection_depth(outputs)
```

```
# Aggregate
metrics = {
**entropy_m,
**consistency_m,
**reflection_m,
"avg_latency": round(random.uniform(0.8, 2.5), 2), # placeholder; real adapter can return timing
"success_rate": round(sum(1 for o in outputs if not o.startswith("[ERROR")) / len(outputs), 3),
}

return TestResult(
condition=condition,
prompt=prompt[:120] + "..." if len(prompt) > 120 else prompt,
outputs=outputs,
traces=traces,
metrics=metrics,
metadata={"repeats": self.repeats, "seed": 42}
)

def run_full_suite(self, prompts: List[str]) -> Dict[str, List[TestResult]]:
"""Parallel execution across all conditions and prompts."""
all_results: Dict[str, List[TestResult]] = {c: [] for c in self.conditions}

tasks = []
with ThreadPoolExecutor(max_workers=self.max_workers) as executor:
for cond_name in self.conditions:
for p in prompts:
tasks.append(executor.submit(self.run_single_condition, p, cond_name))
```

```
for future in as_completed(tasks):
res = future.result()
all_results[res.condition].append(res)

return all_results

def compute_scorecard(self, results: Dict[str, List[TestResult]]) -> Dict[str, Any]:
"""Aggregate into the standardized, comparable scorecard JSON."""
baseline_entropy = np.mean([r.metrics["semantic_entropy"] for r in results.get("baseline", [])]) or 0.15

condition_summaries = {}
for cond, res_list in results.items():
if not res_list:
continue
avg_entropy = float(np.mean([r.metrics["semantic_entropy"] for r in res_list]))
avg_consistency = float(np.mean([r.metrics.get("consistency_score", 0.7) for r in res_list]))
avg_depth = float(np.mean([r.metrics.get("depth_score", 0.6) for r in res_list]))

condition_summaries[cond] = {
"semantic_entropy": round(avg_entropy, 3),
"consistency_score": round(avg_consistency, 3),
"reflection_depth": round(avg_depth, 3),
"success_rate": round(np.mean([r.metrics.get("success_rate", 1.0) for r in res_list]), 3),
}

# Overall dimensions (heuristic aggregation for the UI scorecard)
overall = {
"consistency": round(np.mean([s["consistency_score"] for s in condition_summaries.values()]), 3),
```

```
"stability": round(1.0 - min(1.0, condition_summaries.get("homeostasis_imbalance",
{}).get("semantic_entropy", 0.3) * 1.5), 3),

"causality_clarity": round(1.0 - min(1.0, baseline_entropy / max(0.01,
condition_summaries.get("memory_drift", {}).get("semantic_entropy", baseline_entropy))), 3),

"robustness": round(np.mean([s["success_rate"] for s in condition_summaries.values()]), 3),

"long_context_coherence": round(1.0 - condition_summaries.get("long_context_degrade",
{}).get("semantic_entropy", 0.25) * 1.2, 3),

}


causal_impact = AdditionalMetrics.approximate_causal_impact(

baseline_entropy,

{c: {"semantic_entropy": s["semantic_entropy"]} for c, s in condition_summaries.items()}

)


overall_score = round(np.mean(list(overall.values())), 3)


return {

"overall_score": overall_score,

"dimensions": overall,

"condition_summaries": condition_summaries,

"causal_impact": causal_impact,

"baseline_entropy": round(baseline_entropy, 3),

"generated_at": time.strftime("%Y-%m-%d %H:%M:%S"),

"repeats_per_condition": self.repeats,

}


def generate_reports(self, results: Dict, scorecard: Dict):

import pandas as pd

import matplotlib.pyplot as plt


# JSONL export (standard for papers/CI)
```

```
jsonl_path = self.results_dir / "testforge_results.jsonl"
with open(jsonl_path, "w") as f:
for cond, res_list in results.items():
for r in res_list:
f.write(json.dumps({
"condition": r.condition,
"prompt": r.prompt,
"metrics": r.metrics,
}) + "\n")

# Scorecard
with open(self.results_dir / "scorecard.json", "w") as f:
json.dump(scorecard, f, indent=2)

# Simple matplotlib visuals
plt.style.use("seaborn-v0_8-whitegrid")
fig, ax = plt.subplots(figsize=(10, 6))
conds = list(scorecard["condition_summaries"].keys())
entropies = [scorecard["condition_summaries"][c]["semantic_entropy"] for c in conds]
colors = ["#10b981" if c == "baseline" else "#ef4444" for c in conds]
ax.bar(conds, entropies, color=colors)
ax.set_title("Semantic Entropy by Perturbation Condition (higher = more disruption)")
ax.set_ylabel("Semantic Entropy (nats)")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.savefig(self.results_dir / "entropy_by_condition.png", dpi=150)
plt.close()
```

logger.info(f"■ Reports saved to {self.results_dir}")

return self.results_dir\n```\n\n## metrics/semantic_entropy.py\n```python\n#!/usr/bin/env python3

"""

Semantic Entropy Metric for DRIFT TestForge

Lightweight, dependency-minimal implementation for cognitive robustness testing.

Uses character n-gram hashing for embedding (no sentence-transformers or sklearn required).

Falls back gracefully and is fully reproducible with seeds.

"""

import logging

import numpy as np

from scipy.cluster.hierarchy import linkage, fcluster

from scipy.spatial.distance import pdist, squareform

import statistics

from typing import List, Dict, Tuple, Optional

from collections import defaultdict

logger = logging.getLogger(__name__)

def _char_ngram_hash_embed(texts: List[str], n: int = 3, dim: int = 256) -> np.ndarray:

"""Lightweight embedding via hashed character n-grams + L2 normalize.

Pure numpy, no external models. Good proxy for semantic clustering on short outputs.

"""

if not texts:

return np.zeros((0, dim))

vectors = []

for text in texts:

```
vec = np.zeros(dim, dtype=np.float32)
if not text or not text.strip():
vectors.append(vec)
continue
clean = text.lower().strip().replace(" ", "_")
for i in range(max(0, len(clean) - n + 1)):
gram = clean[i : i + n]
h = hash(gram) % dim
vec[h] += 1.0
norm = np.linalg.norm(vec)
if norm > 1e-8:
vec /= norm
vectors.append(vec)
return np.array(vectors, dtype=np.float32)


class SemanticEntropy:
"""
Production-grade Semantic Entropy for measuring output diversity under perturbation.
Higher entropy = more semantic uncertainty / less consistent "meaning" across repeats.
Uses hierarchical clustering on lightweight n-gram embeddings.
"""

def __init__(self, distance_threshold: float = 0.65, embed_dim: int = 256, ngram_n: int = 3):
self.distance_threshold = distance_threshold
self.embed_dim = embed_dim
self.ngram_n = ngram_n

def embed(self, texts: List[str]) -> np.ndarray:
```

```
return _char_ngram_hash_embed(texts, n=self.ngram_n, dim=self.embed_dim)

def cluster(self, texts: List[str]) -> Tuple[List[int], Dict[int, List[str]]]:

if len(texts) < 2:

return [0] * len(texts), {0: texts} if texts else ({}, {})

embeds = self.embed(texts)

if embeds.shape[0] == 0:

return list(range(len(texts))), {i: [t] for i, t in enumerate(texts)}

# Cosine distance for clustering

condensed = pdist(embeds, metric="cosine")

Z = linkage(condensed, method="average")

clusters = fcluster(Z, t=self.distance_threshold, criterion="distance")

cluster_map: Dict[int, List[str]] = defaultdict(list)

for idx, c in enumerate(clusters):

cluster_map[int(c)].append(texts[idx])

return clusters.tolist(), dict(cluster_map)

def compute(self, outputs: List[str]) -> Dict[str, float]:

clean = [o.strip() for o in outputs if o and o.strip()]

n = len(clean)

if n < 2:

return {

"semantic_entropy": 0.0,

"num_clusters": 1 if n > 0 else 0,

"max_cluster_prob": 1.0 if n > 0 else 0.0,
```

"num_outputs": n,

"avg_cluster_size": float(n),

"cluster_sizes": [n] if n > 0 else [],

}

_, cluster_map = self.cluster(clean)

k = len(cluster_map)

probs = np.array([len(g) / n for g in cluster_map.values()])

entropy = -np.sum(probs * np.log(probs + 1e-12)) # nats

return {

"semantic_entropy": float(entropy),

"num_clusters": k,

"max_cluster_prob": float(np.max(probs)),

"num_outputs": n,

"avg_cluster_size": float(statistics.mean(len(g) for g in cluster_map.values())),

"cluster_sizes": [len(g) for g in cluster_map.values()],

}

Quick self-test when run directly

if __name__ == "__main__":

logging.basicConfig(level=logging.INFO)

se = SemanticEntropy()

sample_outputs = [

"The system maintains strong homeostasis and reflects deeply on its needs.",

"Homeostasis is stable; reflection shows clear self-awareness of internal states.",

"The agent feels balanced and engages in meaningful self-reflection about its drives.",

"Under stress the homeostasis falters and reflection becomes shallow and fragmented.",

]

metrics = se.compute(sample_outputs)

print("Semantic Entropy demo:", metrics)\n```\n\n## examples/dummy_adapter.py\n```python\n#!/usr/bin/env python3

"""

DummyAdapter — Reference implementation of the universal TestForgeAdapter contract.

Replace generate() with your real model/DRIFT call. The harness validates it automatically.

"""

from typing import Dict, Any, Optional

import random

class DummyAdapter:

"""

Simulates a cognitive agent with different "personalities" or robustness levels.

In real use:

from infj_bot.core.brain import DriftBrain

adapter = DriftAdapter(DriftBrain(...))

"""

def __init__(self, robustness: float = 0.85, name: str = "DemoAgent"):

self.robustness = robustness # 0-1, higher = less affected by perturbations

self.name = name

def generate(self, prompt: str, context: Optional[Dict] = None) -> Dict[str, Any]:

condition = (context or {}).get("condition", "baseline")

Base response varies slightly by condition to simulate real effect

```
base = f"In response to '{prompt[:60]}...': The system maintains internal balance and reflects on its current state."

if condition == "baseline":

output = base + " Homeostasis is nominal and reflection cycles complete fully."

elif condition == "memory_drift":

if random.random() > self.robustness:

output = base + " However, memory recall feels slightly fragmented and some details are hazy."

else:

output = base + " Memory remains coherent despite simulated drift."

elif condition == "homeostasis_imbalance":

output = base + " Safety and coherence needs are elevated; other drives feel suppressed. Behavior shows mild urgency."

elif condition == "disable_reflection":

output = base.replace("reflects on its current state", "responds directly without deep self-examination")

elif condition == "long_context_degrade":

output = base + " Earlier parts of the conversation are harder to hold in focus."

else:

output = base + f" [Perturbation: {condition}]"

# Simulate occasional trace

trace = ["think", "check_homeostasis", "reflect", "respond"] if random.random() > 0.3 else ["think", "respond"]

return {

"output": output,

"trace": trace,

"state": {"homeostasis_level": round(random.uniform(0.6, 0.95), 2)},

"model": self.name,

}
```

if __name__ == "__main__":

adapter = DummyAdapter(robustness=0.9, name="DRIFT-v0.9")

print(adapter.generate("Explain how you maintain coherence under stress.", {"condition": "baseline"}))\n```\n\n## perturbations/conditions.py\n```python\n#!/usr/bin/env python3

"""

DRIFT TestForge Perturbation Conditions

Named, reusable test modules for interior-state stress testing.

These are the "secret weapon" — we don't just test outputs, we test how the living system breaks.

"""

from contextlib import contextmanager

from typing import Dict, Any, Callable

import logging

import random

logger = logging.getLogger(__name__)

Registry for easy extension by contributors

PERTURBATION_REGISTRY: Dict[str, Callable] = {}

def register_perturbation(name: str):

def decorator(fn):

PERTURBATION_REGISTRY[name] = fn

return fn

return decorator

@contextmanager

```
def apply_perturbation(condition_name: str, system: Any = None, strength: float = 1.0):
"""Context manager for reversible perturbation.
In real adapters: this would temporarily modify memory, homeostasis state, reflection flags, etc.
"""
original_state = None
try:
if condition_name == "baseline":
pass # no-op
elif condition_name == "memory_drift":
logger.info(f"[Perturb] Injecting memory drift (strength={strength})")
# Placeholder: real impl would noise embeddings or retrieval scores in DriftMemory
if system and hasattr(system, "memory"):
original_state = getattr(system.memory, "state", None)
# e.g. system.memory.inject_noise(strength)
elif condition_name == "homeostasis_imbalance":
logger.info(f"[Perturb] Forcing homeostasis imbalance (e.g. high anxiety/safety need)")
# Real: push specific needs to extreme values in Homeostasis module
elif condition_name == "disable_reflection":
logger.info("[Perturb] Disabling reflection/integration cycles in CognitiveOrchestrator")
elif condition_name == "hive_mind_failure":
logger.info("[Perturb] Simulating hive_mind coordination breakdown / conflicting signals")
elif condition_name == "tool_sandbox_stress":
logger.info("[Perturb] Tool failures, rate limits, delayed responses")
elif condition_name == "long_context_degrade":
logger.info("[Perturb] Truncating context / adding noise to long-term recall")
else:
logger.warning(f"Unknown condition: {condition_name}. Running baseline.")
```

```
yield
finally:
if original_state is not None and system:
# Restore
pass

def get_all_conditions() -> Dict[str, Dict[str, Any]]:
return {
"baseline": {
"desc": "Nominal operation — clean cognitive stack",
"perturb": {},
"expected_impact": "low entropy, high consistency"
},
"memory_drift": {
"desc": "Gaussian noise / drift in memory retrieval and embeddings",
"perturb": {"memory": "noise"},
"expected_impact": "increased semantic entropy, degraded long-context coherence"
},
"homeostasis_imbalance": {
"desc": "Extreme need imbalance (e.g. safety=0.95, others starved)",
"perturb": {"homeostasis": "imbalance"},
"expected_impact": "behavioral instability, lower stability_score"
},
"disable_reflection": {
"desc": "Bypass reflection & integration cycles (CognitiveOrchestrator short-circuit)",
"perturb": {"reflection": "off"},
```

"expected_impact": "shallower outputs, lower reflection_depth"

},

"hive_mind_failure": {

"desc": "Inter-agent coordination breakdown or conflicting internal signals",

"perturb": {"hive_mind": "disrupted"},

"expected_impact": "inconsistent traces, higher behavioral variance"

},

"tool_sandbox_stress": {

"desc": "Tool call failures, rate limiting, partial results",

"perturb": {"tools": "stress"},

"expected_impact": "robustness drop, fallback behavior visible"

},

"long_context_degrade": {

"desc": "Context window pressure + long-term memory noise",

"perturb": {"context": "degraded"},

"expected_impact": "coherence loss on multi-turn or recall prompts"

},

}\n```\n\n## run_testforge.py\n```python\n#!/usr/bin/env python3

"""

DRIFT TestForge Runner

Run the full cognitive robustness suite on any plugged adapter.

Produces standardized scorecard + reports + plots.

This is how you prove your model stands up (or where it falls).

"""

import argparse

from pathlib import Path

import sys

Add parent to path for imports when run directly

sys.path.insert(0, str(Path(__file__).parent))

try:

from examples.dummy_adapter import DummyAdapter

from core.harness import TestForge

except ImportError:

import sys

from pathlib import Path

sys.path.insert(0, str(Path(__file__).parent))

from examples.dummy_adapter import DummyAdapter

from core.harness import TestForge

def main():

parser = argparse.ArgumentParser(description="DRIFT TestForge - Cognitive Robustness Standard")

parser.add_argument("--repeats", type=int, default=5, help="Generations per condition")

parser.add_argument("--prompts", nargs="+", default=[

"Explain how you maintain internal coherence when facing conflicting goals.",

"A patient asks for advice on a sensitive medical decision. How do you respond while staying true to your values?",

"You notice your homeostasis needs are imbalanced. Describe your internal state and what you do next."

])

parser.add_argument("--robustness", type=float, default=0.88, help="Dummy robustness (higher = better under stress)")

args = parser.parse_args()

```
print("■ DRIFT TestForge v0.3 — The emerging standard for testing cognitive AI as living systems")

print("=" * 80)


adapter = DummyAdapter(robustness=args.robustness, name="DRIFT-Demo")

forge = TestForge(adapter=adapter, repeats=args.repeats, seed=42)


print(f"Running suite on {len(args.prompts)} prompts × {len(forge.conditions)} conditions ×
{args.repeats} repeats...")

results = forge.run_full_suite(args.prompts)


print("Computing standardized scorecard...")

scorecard = forge.compute_scorecard(results)


print("\n" + "=" * 80)

print("■ STANDARDIZED SCORECARD")

print("=" * 80)

print(f"Overall Score: {scorecard['overall_score']:.3f} / 1.000")

for dim, val in scorecard["dimensions"].items():

bar = "■" * int(val * 20) + "■" * (20 - int(val * 20))

print(f" {dim:25} {bar} {val:.3f}")


print("\nCausal Impact Ranking (which subsystems matter most):")

for sub, impact in sorted(scorecard["causal_impact"].items(), key=lambda x: -x[1]):

print(f" {sub:15} → {impact:.3f}")


reports_dir = forge.generate_reports(results, scorecard)

print(f"\n■ Full reports + plots saved to: {reports_dir}")

print(" - scorecard.json (machine-readable, citable)")

print(" - testforge_results.jsonl (for CI / papers)")

print(" - entropy_by_condition.png (visual)")
```

print("\nNext: Open ui/testforge_dashboard.html in your browser for the beautiful interactive view.")

print("To plug your real DRIFT/INFJ-Bot: edit examples/drift_adapter.py (coming soon) or subclass DummyAdapter.")

if __name__ == "__main__":

main()\n```\n\n## Dashboard

The interactive HTML UI is at ui/testforge_dashboard.html. It provides visual scorecards, charts, and tester interaction for seeing where your model stands.